Selenium By Locators Guide

Oct 30, 2026 · @Blue

Setup and how By works

Every locator follows one pattern: `driver.find_element(By.STRATEGY, "value")`. The strategy says how to search, and the value says what to look for.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://webscraper.io/test-sites/pagination")

element = driver.find_element(By.TAG_NAME, "h3")
print(element.text)

driver.quit()
```

What each line does:

- `from selenium import webdriver`: imports the module that starts and controls a browser.
- `from selenium.webdriver.common.by import By`: imports the `By` class, which holds the eight strategy names.
- `webdriver.Chrome()`: opens a Chrome window. Selenium 4.6 and newer downloads the matching driver for you.
- `driver.get(url)`: loads the page and waits for the load to finish.
- `driver.find_element(By.TAG_NAME, "h3")`: returns the first matching element, or raises `NoSuchElementException` if there is none.
- `element.text`: the visible text of that element.
- `driver.quit()`: closes the browser and ends the session. Always call it, or Chrome processes stay open in the background.

The eight strategies in `By`:

Strategy	Searches by
`By.ID`	the `id` attribute
`By.NAME`	the `name` attribute
`By.CLASS_NAME`	one class inside the `class` attribute
`By.TAG_NAME`	the tag, like `div` or `a`
`By.LINK_TEXT`	the full text of an `<a>` link
`By.PARTIAL_LINK_TEXT`	part of the text of an `<a>` link
`By.CSS_SELECTOR`	a CSS selector
`By.XPATH`	an XPath expression

By.ID

`By.ID` finds the element whose `id` attribute matches exactly. It is the fastest and most reliable locator because an id should be unique on the page.

HTML:

```
<input id="search-box" type="text" placeholder="Search cars">
<button id="submit-btn">Search</button>
```

Python:

```
box = driver.find_element(By.ID, "search-box")
box.send_keys("Ferrari")
driver.find_element(By.ID, "submit-btn").click()
```

- `By.ID, "search-box"`: matches the `<input>` because its `id` is exactly `search-box` (case-sensitive).
- `box.send_keys("Ferrari")`: types the text into the field, like a keyboard would.
- `.click()`: clicks the button found by its id.

Watch out: some sites generate ids that change on every load, like `id="input_83412"`. Don't use those. Pick another locator instead.

By.NAME

`By.NAME` finds elements by their `name` attribute. It is common on form fields, because the browser uses `name` when it submits the form.

HTML:

```
<form action="/search">
  <input name="q" type="text">
  <input name="fuel" type="radio" value="petrol">
  <input name="fuel" type="radio" value="diesel">
</form>
```

Python:

```
field = driver.find_element(By.NAME, "q")
field.clear()
field.send_keys("BMW")

radios = driver.find_elements(By.NAME, "fuel")
radios[1].click()
```

- `By.NAME, "q"`: matches the text input whose `name` is `q`.
- `field.clear()`: empties the field first, in case it already has text.
- `field.send_keys("BMW")`: types into the field.
- `find_elements(By.NAME, "fuel")`: returns a list of both radio buttons, because they share one name.
- `radios[1].click()`: clicks the second one (index 1, so "diesel").

Watch out: names are not required to be unique. `find_element` returns only the first match, so use `find_elements` when several elements share a name.

By.CLASS_NAME

`By.CLASS_NAME` finds elements that have a given CSS class. Many elements usually share a class, so it is most useful with `find_elements` to collect repeated items like product cards.

HTML:

```
<div class="card product">
  <h4 class="title">Ferrari F40 1984</h4>
  <span class="price">USD 277 672</span>
</div>
<div class="card product">
  <h4 class="title">BMW E24 635CSi 1954</h4>
  <span class="price">USD 18 457</span>
</div>
```

Python:

```
titles = driver.find_elements(By.CLASS_NAME, "title")
prices = driver.find_elements(By.CLASS_NAME, "price")

for t, p in zip(titles, prices):
    print(t.text, p.text)
```

- `By.CLASS_NAME, "title"`: matches every element whose class list contains `title`.
- `find_elements`: returns a list of all matches (an empty list if none).
- `zip(titles, prices)`: pairs the first title with the first price, the second with the second, and so on.
- `.text`: the visible text of each element.

Watch out: pass only one class name, with no spaces. `By.CLASS_NAME, "card product"` fails, because the value is treated as a single class. To require two classes on the same element, use CSS: `By.CSS_SELECTOR, ".card.product"`.

By.TAG_NAME

`By.TAG_NAME` finds elements by their HTML tag, like `table`, `tr`, `td`, `a`, or `img`. It works well for tables and lists, and for searching inside an element you already found.

HTML:

```
<table>
  <tr><th>Brand</th><th>Year</th></tr>
  <tr><td>Ferrari</td><td>1984</td></tr>
  <tr><td>BMW</td><td>1954</td></tr>
</table>
```

Python:

```
rows = driver.find_elements(By.TAG_NAME, "tr")

for row in rows:
    cells = row.find_elements(By.TAG_NAME, "td")
    print([c.text for c in cells])
```

- `find_elements(By.TAG_NAME, "tr")`: returns every table row on the page.
- `row.find_elements(...)`: called on an element instead of `driver`, so it searches only inside that row.
- `[c.text for c in cells]`: a list comprehension that collects the text of each cell.
- The header row prints `[]`, because it holds `<th>` cells and no `<td>` cells.

Watch out: a tag like `div` or `a` matches hundreds of elements on a real page. Use `By.TAG_NAME` on a small scope, or switch to a CSS selector.

By.LINK_TEXT and By.PARTIAL_LINK_TEXT

Both locators work only on `<a>` links and match the text the user sees. `LINK_TEXT` needs the whole text to match exactly. `PARTIAL_LINK_TEXT` needs only part of it.

HTML:

```
<a href="/test-sites/pagination/BMW">BMW</a>
<a href="/test-sites/pagination?page=2">2</a>
<a href="/test-sites/product/ferrari-f40-1984-c005">Ferrari F40 1984</a>
```

Python:

```
bmw = driver.find_element(By.LINK_TEXT, "BMW")
print(bmw.get_attribute("href"))
bmw.click()

ferrari = driver.find_element(By.PARTIAL_LINK_TEXT, "Ferrari")
print(ferrari.text)
```

- `By.LINK_TEXT, "BMW"`: matches the link whose full text is exactly `BMW` (case-sensitive).
- `get_attribute("href")`: reads the link's URL. Useful for collecting links to visit later.
- `.click()`: follows the link, like a user clicking it.

- `By.PARTIAL_LINK_TEXT, "Ferrari"`: matches any link whose text contains `Ferrari`, so it finds "Ferrari F40 1984".

Watch out: `PARTIAL_LINK_TEXT` can match several links, and `find_element` returns only the first. Use `find_elements` to get them all. Neither locator works if the link text is an image or is hidden by CSS.

By.CSS_SELECTOR

`By.CSS_SELECTOR` takes the same selector syntax you would write in a stylesheet. It covers ids, classes, attributes, and nesting in one locator, so it is the most flexible everyday choice.

All the patterns below use this HTML:

```
<div id="listing">
  <div class="card featured" data-status="reserved">
    <h3><a class="title" href="/product/bmw-e24-1954-c002">BMW E24 635CSi
1954</a></h3>
    <p class="price">USD 18 457</p>
  </div>
  <div class="card" data-status="sold">
    <h3><a class="title" href="/product/nissan-300zx-1966-c004">Nissan 300ZX
Z31 1966</a></h3>
    <p class="price">USD 104 195</p>
  </div>
</div>
```

Goal	Selector	Matches
By id	`#listing`	the outer div
By class	`.price`	both prices
Tag and class	`p.price`	both prices (only `<p>` tags)
Two classes on one element	`.card.featured`	the first card only
Direct child	`#listing > .card`	both cards
Any descendant	`.card a`	both links
Attribute equals	`[data-status="sold"]`	the second card

Goal	Selector	Matches
Attribute starts with	`a[href^="/product/bmw"]`	the first link
Attribute contains	`a[href*="nissan"]`	the second link
Position	`.card:nth-child(2)`	the second card

Python:

```
cards = driver.find_elements(By.CSS_SELECTOR, "#listing > .card")

for card in cards:
    title = card.find_element(By.CSS_SELECTOR, "h3 a.title").text
    price = card.find_element(By.CSS_SELECTOR, "p.price").text
    link = card.find_element(By.CSS_SELECTOR,
"a.title").get_attribute("href")
    status = card.get_attribute("data-status")
    print(title, price, status, link)
```

- `"#listing > .card"`: selects the cards that are direct children of the element with id `listing`.
- `card.find_element(...)`: searches only inside that card, so each price belongs to the right title.
- `"h3 a.title"`: an `<a>` with class `title` that sits inside an `<h3>`.
- `.text`: the visible text of the element.
- `get_attribute("href")`: the link's full URL.
- `card.get_attribute("data-status")`: reads the card's own attribute, which here is `reserved` or `sold`.

Watch out: CSS cannot select an element by its text, and it cannot go from a child up to its parent. Use XPath for those.

By.XPATH

`By.XPATH` navigates the page like a file path. It is the most powerful locator: it can match by text and walk up to a parent or sideways to a sibling. It is also the easiest to write badly, so keep expressions short.

All the patterns below use the same HTML as the CSS section:

```
<div id="listing">
  <div class="card featured" data-status="reserved">
    <h3><a class="title" href="/product/bmw-e24-1954-c002">BMW E24 635CSi
1954</a></h3>
    <p class="price">USD 18 457</p>
  </div>
  <div class="card" data-status="sold">
    <h3><a class="title" href="/product/nissan-300zx-1966-c004">Nissan 300ZX
Z31 1966</a></h3>
    <p class="price">USD 104 195</p>
  </div>
</div>
```

Goal	XPath	Matches
Any tag, anywhere	`//p`	both prices
Attribute equals	`//div[@data-status="sold"]`	the second card
Class contains	`//div[contains(@class,"featured")]`	the first card
Exact text	`//a[normalize-space()="BMW E24 635CSi 1954"]`	the first link
Text contains	`//a[contains(text(),"Nissan")]`	the second link
Attribute starts with	`//a[starts-with(@href,"/product/bmw")]`	the first link
Parent	`//p[@class="price"]/..`	both cards
Position (starts at 1)	`(//div[contains(@class,"card")])[2]`	the second card
Price of a given title	`//a[contains(text(),"Nissan")]/ancestor::div[contains(@class,"card")]//p[@class="price"]`	the second price

Python:

```
price = driver.find_element(
    By.XPATH,

'//a[contains(text(),"Nissan")]/ancestor::div[contains(@class,"card")]//p[@cl
ass="price"]'
).text
print(price)

cards = driver.find_elements(By.XPATH, '//div[contains(@class,"card")]')
for card in cards:
    print(card.find_element(By.XPATH, './/p[@class="price"]').text)
```

- `//a[contains(text(),"Nissan")]`: finds the `<a>` whose text includes `Nissan`.
- `/ancestor::div[contains(@class,"card")]`: climbs up to the card that contains that link.
- `//p[@class="price"]`: from that card, goes down to its price paragraph.
- `'...'` around the XPath: single quotes outside, so the double quotes inside the expression need no escaping.
- `.//p[...]` (with the leading dot): searches only inside the current `card`.

Watch out:

- A path starting with `//` searches the whole page, even when you call `find_element` on an element. Start with `.//` to stay inside it.
- `@class="card"` is an exact match, so it misses `class="card featured"`. Use `contains(@class,"card")`.
- Avoid long absolute paths copied from DevTools, like `/html/body/div[2]/div[1]/p`. They break when the layout changes.

find_element vs find_elements, and which locator to choose

Use `find_element` when you expect one match and `find_elements` when you expect many. The difference shows up most when nothing matches.

	`find_element`	`find_elements`
Returns	the first match	a list of all matches
When nothing matches	raises `NoSuchElementException`	returns an empty list `[]`

	`find_element`	`find_elements`
Typical use	one button, one field, one title	cards, rows, links

Which locator to pick:

Situation	Use
The element has a stable, unique id	`By.ID`
A form field	`By.NAME`
Nested structure, classes, or attributes	`By.CSS_SELECTOR`
A table's rows and cells	`By.TAG_NAME` or `By.CSS_SELECTOR`
A navigation link with known text	`By.LINK_TEXT` or `By.PARTIAL_LINK_TEXT`
Match by text, or move to a parent or sibling	`By.XPATH`

A good order to try them in is ID, then NAME, then CSS_SELECTOR, then XPATH.

On pages that load content with JavaScript, the element may not exist yet when you search for it. Wait for it instead of using `time.sleep`:

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

wait = WebDriverWait(driver, 10)
button = wait.until(EC.element_to_be_clickable((By.ID, "submit-btn")))
button.click()

cards = wait.until(
    EC.presence_of_all_elements_located((By.CSS_SELECTOR, ".card"))
)
```

- `WebDriverWait(driver, 10)`: creates a waiter that will keep checking for up to 10 seconds.
- `wait.until(...)`: checks repeatedly and returns as soon as the condition is true. If 10 seconds pass first, it raises `TimeoutException`.
- `EC.element_to_be_clickable(...)`: true when the element is visible and enabled.

- `EC.presence_of_all_elements_located(...)`: true when at least one element matches, and returns the list of them.
- The locator goes in as a tuple, `(By.ID, "submit-btn")`, with double parentheses. That is the most common mistake here.

Practice on the pagination test site

The pagination test site is a fake classic-car marketplace with 17 pages and brand links in a sidebar. The locators below are starting points, not tested against the live HTML. Right-click a card and choose Inspect to confirm the real tags and classes before you rely on them.

To get	Try	Why
A brand page (BMW, Ferrari, ...)	`By.LINK_TEXT, "BMW"`	the link text is exactly the brand name
A page number	`By.LINK_TEXT, "2"`	the link text is the number
The next page	`By.LINK_TEXT, "›"`	the link text is the arrow
Car title and detail URL	`By.CSS_SELECTOR, "h3 a"`	each title looks like a heading that wraps a link
Status (Available, Reserved, Sold)	`By.XPATH, '//*[contains(text(),"Reserved")]'`	the label is identified by its text
Price, year, mileage, country	inspect first, then use a class with CSS	the text is like `USD 228 511` and `Year: 1955`, so find the class that wraps each one

First try: collect the titles on page 1 and open page 2.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://webscraper.io/test-sites/pagination")

titles = driver.find_elements(By.CSS_SELECTOR, "h3 a")
print(len(titles))
for t in titles:
    print(t.text, "->", t.get_attribute("href"))

driver.find_element(By.LINK_TEXT, "2").click()
print(driver.current_url)

driver.quit()
```

- `find_elements(By.CSS_SELECTOR, "h3 a")`: collects every link inside an `<h3>`. A page should give 6, so check that `len(titles)` agrees.
- `t.text` and `t.get_attribute("href")`: the car's name and the URL of its detail page.
- `find_element(By.LINK_TEXT, "2").click()`: clicks the page 2 link.
- `driver.current_url`: the address the browser is on now. It should end in `?page=2`.

Then try walking all 17 pages by following the next link until it is gone:

```
import time

all_titles = []
while True:
    for a in driver.find_elements(By.CSS_SELECTOR, "h3 a"):
        all_titles.append(a.text)

    next_links = driver.find_elements(By.LINK_TEXT, "›")
    if not next_links:
        break
    next_links[0].click()
    time.sleep(1)

print(len(all_titles))
```

- `all_titles = []`: a list that collects the titles from every page.
- `while True:`: repeats until the `break` runs.

- `find_elements(By.LINK_TEXT, "›")`: returns a list, so on the last page it returns `[]` instead of raising an error. On page 1 the `‹` is plain text, not a link, so `›` is probably the same on page 17.
- `if not next_links: break`: stops the loop when there is no next link.
- `next_links[0].click()`: goes to the next page.
- `time.sleep(1)`: a one-second pause between pages, so you don't hammer the server.

The pages also follow the pattern `?page=N`, so a second approach is to skip clicking and loop `driver.get(f".../pagination?page={n}")` for `n` from 1 to 17. Try both and compare how fast and how stable each one is.